PEMBELAJARAN MESIN (*MACHINE LEARNING*)

Praktikum 3 Ektraksi Fitur

Dosen Pengampu Mata Kuliah

Dr. Ely Setyo Astuti, S.T., M.T.

Disusun Oleh :

Siti Nikmatus Sholihah 244107020014

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Praktikum 3a
Pra Pengolahan Data - Ektraksi Fitur (Data Terstruktur)

Langkah 1 - Load Data

Import library yang dibutuhkan kemudian, load data kedalam DataFrame Pandas.

```
[1]
import pandas as pd

data = 'Titanic-Dataset2.csv' # path dataset
df = pd.read_csv(data)
```
✓ 4.9s

*jika menggunakan colab, kita harus upload datanya dulu ke colab, baru kemudian bisa diakses

Tampilkan data teratas dengan perintah df.head()

```
[2]
df.head()
```
✓ 0.0s

Maka akan muncul tampilan seperti dibawah ini

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Langkah 2 - Pengecekan Data

Lakukan pengecekan informasi mengenai data dengan menggunakan df.info()

```
[3]
df.info()
```
✓ 0.0s

Perhatikan hasil dari perintah yang telah dijalankan.

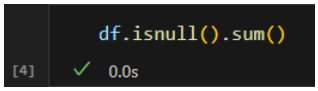
```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

Dari df.info() kita dapat mengetahui jumlah data untuk setiap kolom. Selain itu, kita mengetahui bahwa,

- "Age" berjenis float64
- "Cabin" berjenis object
- "Embarked" berjenis object

Pada Pandas, object sering kali mewaliki tipe data string. Kita harus cermat dalam hal ini. Hal ini berkaitan dengan teknik imputasi yang akan kita gunakan nantinya.

Selanjutnya, untuk lebih mengetahui jumlah data yang hilang untuk setiap kolom, gunakan perintah,



Didapatkan informasi seperti berikut,

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Dari informasi ini, kita dapat mengetahui data yang hilang sejumlah,

- 177 data "Age"
- 687 data "Cabin"
- 2 data "Embarked"

Langkah 3 - Imputasi

Pada langkah ini kita akan melakukan imputasi terdapat data yang hilang pada "Age", "Cabin", dan "Embarked".

Strategi yang akan kita gunakan adalah,

- "Age" --> Dikarenakan "Age" adalah data nominal, maka kita akan menggunakan strategi mean.
- "Cabin" --> "Cabin" merupakan informasi terkait dengan nomor kabin penumpang. Disini kita akan berasumsi bahwa, seluruh penumpang yang tidak memiliki nomor kabin, merupakan penumpang yang tinggal di dek-dek kapal. Sehingga kita akan mensubtitusi data yang hilang dengan informasi "DECK".

- "Embarked" --> "Embarked" merupakan informasi lokasi embarkasi penumpang. Hanya ada 2 data yang hilang disini. Dikarenakan data merupakan data nominal, maka kita dapat menggunakan modus (mode) untuk mensubtitusi data yang hilang.

```
# Age - mean
df['Age'].fillna(value=df['Age'].mean())

# Cabin - "DECK"
df['Cabin'].fillna(value='DECK')

# Embarked - modus
df['Embarked'].fillna(value=df['Embarked'].mode()[0])
```
[11] ✓ 0.0s

Langkah 4 - Validasi Hasil

Lakukan pengecekan kembali dengan df.info(). Selanjutnya, kita juga dapat memastikan kembali apakah nilai yang disikan sudah benar dengan melakukan pencekan data secara langsung. Hasil dari proses imputasi adalah sebagai berikut,

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        891 non-null    str
 11  Embarked     891 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 121.4 KB
```

Langkah 5 - Ekstraksi Fitur

Setelah melakukan Pre-Processing, langkah berikutnya kita akan melakukan Ekstraksi Fitur berdasarkan Jumlah keluarga "Familysize", Gelar Jumlah keluarga "Title", kelompok usia "AgeBin", Tempat tiket "CabinDeck"

```python
import matplotlib.pyplot as plt
import seaborn as sns

# === Ektraksi Fitur === #

# 1. FamilySize: jumlah keluarga di kapal (SibSp + Parch + diri_sendiri)
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

# 2. Title: ambil gelar dari kolom Name (Mr, Mrs, Miss, Master, dll)
df["Title"] = df["Name"].str.extract(r',\s*([^\.]+)\.', expand=False)
title_mapping = {
    "Mlle": "Miss", "Ms": "Miss", "Mme" : "Mrs",
    "Lady": "Rare", "Countess": "Rare", "Capt": "Rare", "Col": "Rare",
    "Don": "Rare", "Dr": "Rare", "Major": "Rare", "Rev": "Rare",
    "Sir": "Rare", "Jonkheer": "Rare", "Dona": "Rare"
}
df['Title'] = df['Title'].replace(title_mapping)

# 3. AgeBin: kelompokkan usid menjadi kategori
df['AgeBin'] = pd.cut(df['Age'], bins=[0, 12, 18, 35, 60, 100],
                      labels=['Child', 'Teen', 'YoungAdult', 'Adult', 'Senior'])

# 4. CabinDeck: ambil huruf pertama dari kolom Cabin
df['CabinDeck'] = df['Cabin'].astype(str).str[0]

# 5. Fare per person: bagi harga tiket dengan jumlah keluarga
df['FarePerson'] = df['Fare'] / df['FamilySize']

print("Hasil Ekstraksi Fitur")
df[['Name', 'Title', 'Age', 'AgeBin', 'FamilySize', 'Cabin', 'CabinDeck', 'FarePerson']].head()
```

[14] ✓ 0.0s

Hasil Ekstraksi Fitur

```
Hasil Ekstraksi Fitur
```

	Name	Title	Age	AgeBin	FamilySize	Cabin	CabinDeck	FarePerson
0	Braund, Mr. Owen Harris	Mr	22.0	YoungAdult	2	DECK	D	3.62500
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	Mrs	38.0	Adult	2	C85	C	35.64165
2	Heikkinen, Miss. Laina	Miss	26.0	YoungAdult	1	DECK	D	7.92500
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	Mrs	35.0	YoungAdult	2	C123	C	26.55000
4	Allen, Mr. William Henry	Mr	35.0	YoungAdult	1	DECK	D	8.05000

Langkah 6 - Visualisasi Hasil Ektraksi Fitur

```python
import seaborn as sns
import matplotlib.pyplot as plt

# === Visualisasi Fitur Hasil Ekstraksi === #

plt.figure(figsize=(12, 6))
sns.barplot(x='Title', y='Survived', data=df),
plt.title('Survival Rate berdasarkan Family Size')
plt.show()

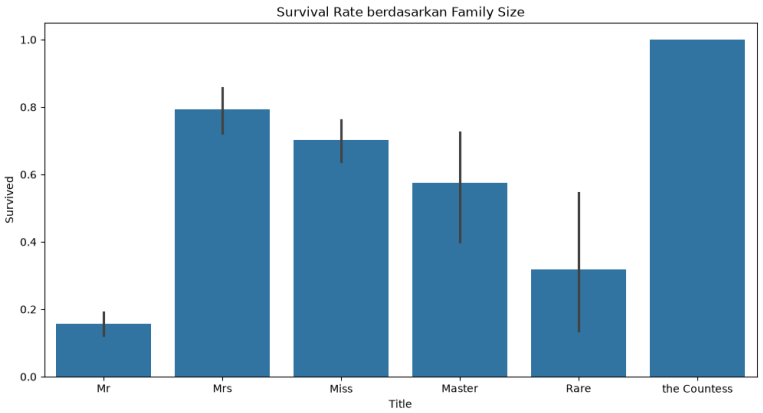
plt.figure(figsize=(12, 6))
sns.barplot(x='Title', y='Survived', data=df,
            order=df.groupby('Title')['Survived'].mean().sort_values().index)
plt.title('Survival Rate berdasarkan Title')
plt.xticks(rotation=45)
plt.show()

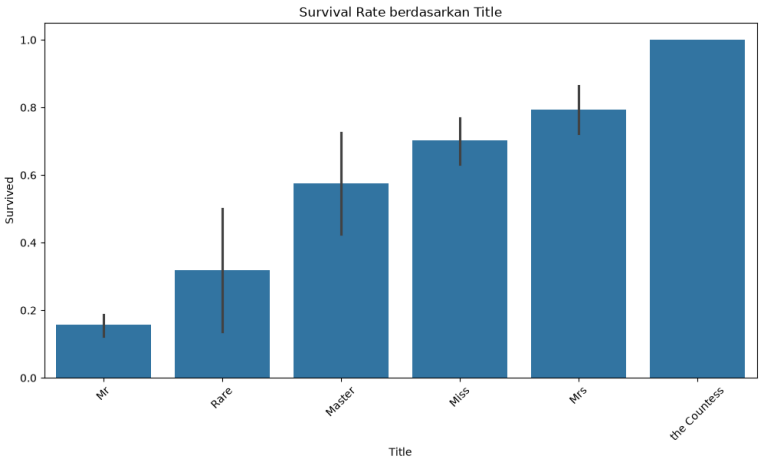
plt.figure(figsize=(12, 6))
sns.barplot(x='AgeBin', y='Survived', data=df,
            order=['Child', 'Teen', 'YoungAdult', 'Adult', 'Senior'])
plt.title('Survival Rate bedasarkan Kelompok Usia (AgeBin)')
plt.show()

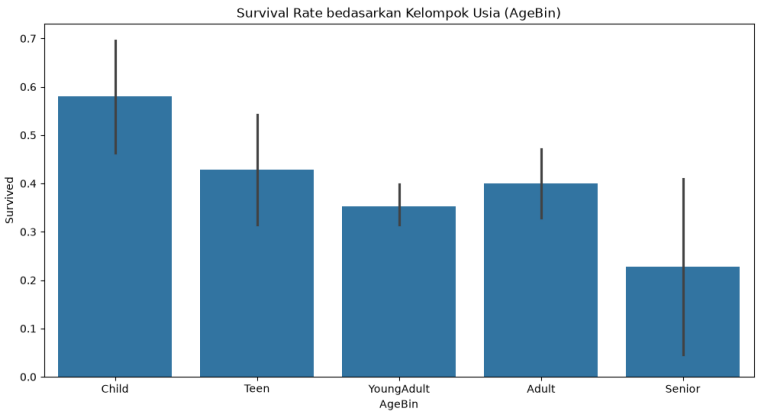
plt.figure(figsize=(12, 6))
sns.barplot(x='CabinDeck', y='Survived', data=df,
                order=df.groupby('CabinDeck')['Survived'].mean().sort_values().index)
plt.title('Survival Rate berdasarkan Cabin Deck')
plt.show()
```

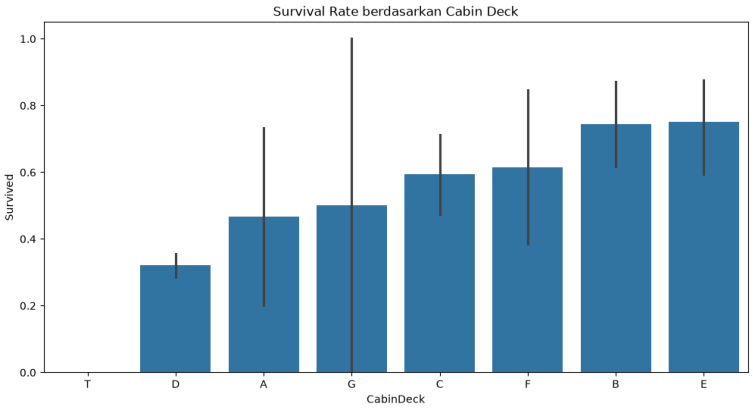
[15] ✓ 2.2s

Hasil dari visualisasi Ektraksi Fitur seperti tampilan di bawah ini


Survival Rate berdasarkan Family Size
Survived
1.0
0.8
0.6
0.4
0.2
0.0
Mr
Mrs
Miss
Master
Rare
the Countess
Title


Survival Rate berdasarkan Title
Survived
1.0
0.8
0.6
0.4
0.2
0.0
Mr
Rare
Master
Miss
Mrs
the Countess
Title


Survival Rate bedasarkan Kelompok Usia (AgeBin)
Survived
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Child
Teen
YoungAdult
Adult
Senior
AgeBin


Survival Rate berdasarkan Cabin Deck
Survived
1.0
0.8
0.6
0.4
0.2
0.0
T
D
A
G
C
F
B
E
CabinDeck

Praktikum 3b
Pra Pengolahan Data - Seleksi Fitur

Pada tahap ini kita akan melakukan Pre- Processing sebelum melakukan seleksi Fitur. Pre processing akan melakukan encoding pada data-data kategorikal, yaitu "Sex" dan "Cabin", imputasi data pada "Age", standarisasi pada fitur "Age". Hal ini perlu dilakukan karena algoritma pembelajaran mesin melakukan proses kalkulasi secara matematis. Salah satu strategi yang bisa di gunakan adalah **Label Encoding**. Label Encoding serupa dengan Ordinal Encoding, bedanya hanya pada proses pengurutan. Label Encoding tidak mengurutkan data terlebih dahulu.

Setelah pre Processing selesai, langkah selanjutnya memilih variabel mana saja yang akan kita gunakan sebagai fitur. Pada bagian dasar teori, kita sudah mempelajari bahwa tidak selalu semua variabel dapat digunakan sebagai fitur. Oleh karena itu, kita dapat memilih fitur yang sekiranya penting untuk tujuan yang ingin kita capai dalam membuat model pembelajaran mesin. Kita akan menggunakan kasus yang sama, yaitu kita kita mengetahui ciri-ciri penumpang kapal Titanic yang selamat. Sehingga, fitur yang kita gunakan adalah,

1. "**Survived**"
2. "**Pclass**"
3. "**Sex**"
4. "**Age**"
5. "**Cabin**"

Kita akan memisahkan variabel-variabel tersebut dari variabel yang lain.

Data yang akan kita gunakan adalah data Titanic yang sudah diperbaiki dengan proses imputasi.

Langkah 0 - Load Library

```
[16]  ✓ 0.0s
```

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Langkah 1 - Load Data

```
[17]  ✓ 0.0s
```

```python
dpath = 'Titanic-Dataset-fixed2.csv'
df = pd.read_csv(dpath)
df.head()
```

Hasil tampilan

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	DECK	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	DECK	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	DECK	S

Langkah 2 - Pre Processing Encoding Sex

Pada tahap ini akan melakukan encoding pada data-data kategorikal, yaitu "Sex"

```
[18]
# --- 2. Encoding fitur 'Sex' ---
df['Sex'] = df['Sex'].map({'male':1, 'female':0})
print('=== Hasil Setelah Encoding "Sex" ===')
print(df[['Sex']].head(), '\n')
```
✓ 0.0s

Langkah 3 - Verifikasi hasil encoding "sex"

```
=== Hasil Setelah Encoding "Sex" ===
   Sex
0    1
1    0
2    0
3    0
4    1
```

Langkah 4- Pre Processing Encoding "Cabin".

Pada tahap ini akan melakukan encoding pada data-data kategorikal, yaitu "Cabin"

```
[21]
# --- 3. Encoding fitur 'Cabin' ---
df['Cabin'] = df['Cabin'].fillna('Unknown')
print("=== Hasil Setelah Fill NaN pada 'Cabin' ===")
print(df[['Cabin']].head(5), "\n")
df['Cabin'] = df['Cabin'].apply(lambda x: x[0] if x != 'Unknown' else 'U')
print("=== Hasil Ambil huruf pertama 'Cabin' (Deck) ===")
print(df[['Cabin']].head(5), "\n")
```
✓ 0.0s

Langkah 5 Verifikasi hasil encoding "Cabin"

```
=== Hasil Setelah Fill NaN pada 'Cabin' ===
  Cabin
0     D
1     C
2     D
3     C
4     D

=== Hasil Ambil huruf pertama 'Cabin' (Deck) ===
  Cabin
0     D
1     C
2     D
3     C
4     D
```

Langkah 6 - Pre Processing Imputasi Data

Pada tahapan ini kita akan melakukan imputasi data pada "Age". Tujuan dari langkah ini adalah untuk demonstrasi proses imputasi data dengan nilai **median**.

```
# --- 4. Standarisasi fitur 'Age' ---
df['Age'] = df[['Age']].fillna(df['Age'].median())
print('=== Hasil Setelah Imputasi Data pada Usia dengan median ===')
print(df[['Age']].head(10), '\n')
```

[22] ✓ 0.0s

Langkah 7 - Verifikasi Hasil Imputasi Data

```
=== Hasil Setelah Imputasi Data pada Usia dengan median ===
        Age
0  22.000000
1  38.000000
2  26.000000
3  35.000000
4  35.000000
5  29.699118
6  54.000000
7   2.000000
8  27.000000
9  14.000000
```

Langkah 8 - Pre Processing Standarisasi

Pada tahapan ini kita akan melakukan standarisasi pada "Age". **Hal ini sebetulnya tidak terlalu diperlukan karena nilai pada "Age" tidak terlalu jauh.** Tujuan dari langkah ini adalah untuk demonstrasi proses standarisasi.

```
scaler = StandardScaler()
df['Age'] = scaler.fit_transform(df[['Age']])
print('=== Hasil Setelah Stansarisasi "Age" ===')
print(df[['Age']].head(10), '\n')
```

[23] ✓ 0.1s

Langkah 9 - Verifikasi Hasil Standarisasi

Cek kembali dengan df.head(). Hasilnya akan seperti pada Gambar dibawah ini.

```
=== Hasil Setelah Stansarisasi "Age" ===
        Age
0 -0.592481
1  0.638789
2 -0.284663
3  0.407926
4  0.407926
5  0.000000
6  1.870059
7 -2.131568
8 -0.207709
9 -1.208115
```

Langkah 10 - Hasil setelah Pre Processing

Pada tahapan ini, akan melakukan pengecekan hasil Pre Processing yang sudah dilakukan yaitu Encoding pada fitur "Sex" dan "Cabin", imputasi data pada "Age", standarisasi pada fitur "Age".

```
# --- 5. Cek Hasil Akir ---
print(' === Hasil Preprocessing (5 baris pertama) ===')
#print(df[['Sex, 'Cabin', 'Age']].head(5))
df.head()
```

[24] ✓ 0.0s

Langkah 11 - Verifikasi Hasil setelah Pre Processing

```
=== Hasil Preprocessing (5 baris pertama) ===
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	1	-0.592481	1	0	A/5 21171	7.2500	D	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	0.638789	1	0	PC 17599	71.2833	C	C
2	3	1	3	Heikkinen, Miss. Laina	0	-0.284663	0	0	STON/O2. 3101282	7.9250	D	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	0.407926	1	0	113803	53.1000	C	S
4	5	0	3	Allen, Mr. William Henry	1	0.407926	0	0	373450	8.0500	D	S

Langkah 12 - Seleksi Fitur

pada tahap ini akan dipilih 5 variabel sebagai fitur penting yaitu fitur **"Survived". "Pclass", "Age", "Sex"**, dan **"Cabin".**

```
df = df[['Survived', 'Pclass', 'Age', 'Sex', 'Cabin']]
df.head()
```

[25] ✓ 0.0s

Verifikasi Hasil Seleksi Fitur

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	D
1	1	1	0.638789	0	C
2	1	3	-0.284663	0	D
3	1	1	0.407926	0	C
4	0	3	0.407926	1	D

Praktikum 3c
Spliting Data

Pada pratikum ini, kita akan mencoba 3 macam strategi pembuatan data latih, validasi, dan uji. Data yang digunakan adalah data kapal Titanic yang sudah kita ekstraksi fiturnya pada pratikum sebelumnya

Random Split

Langkah 1 - Load Data

```
import pandas as pd

df = pd.read_csv('Titanic-Dataset-selected.csv')
df.head()
```

[26] ✓ 0.1s

Hasil load data

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	115
1	1	1	0.638789	0	81
2	1	3	-0.284663	0	115
3	1	1	0.407926	0	55
4	0	3	0.407926	1	115

Langkah 2 - Split Data

```
# Split Data
from sklearn.model_selection import train_test_split

# Split data training dan lainnya
# data lainnya, akan kita split lagi menjadi validasi dan testing.
# Rasio yang akan kita gunakan adaah 8:1:1
df_train, df_unseen = train_test_split(df, test_size=0.2, random_state=0)

#Split lagi antara validasi dan testing
df_val, df_test = train_test_split(df_unseen, test_size=0.5, random_state=0)

# Cek masing-masing ukuran data

print(f'Jumlah data asli: {df.shape[0]}')
print(f'Jumlah data train: {df_train.shape[0]}')
print(f'Jumlah data val: {df_val.shape[0]}')
print(f'Jumlah data test: {df_test.shape[0]}')

# Cek rasio tiap label
print('==========')
print(f'Jumlah label data asli:\n{df.Survived.value_counts()}')
print(f'Jumlah label data train:\n{df_train.Survived.value_counts()}')
print(f'Jumlah label data val:\n{df_val.Survived.value_counts()}')
print(f'Jumlah label data test:\n{df_test.Survived.value_counts}')
```

[28] ✓ 0.0s

Hasilnya adalah sebagai berikut,

```
Jumlah data asli: 891
Jumlah data train: 712
Jumlah data val: 89
Jumlah data test: 90
=========
Jumlah label data asli:
Survived
0    549
1    342
Name: count, dtype: int64
Jumlah label data train:
Survived
0    439
1    273
Name: count, dtype: int64
Jumlah label data val:
Survived
0    53
1    36
Name: count, dtype: int64
Jumlah label data test:
<bound method IndexOpsMixin.value_counts of 40     0
318    1
103    0
712    1
...
8      1
822    0
487    0
Name: Survived, Length: 90, dtype: int64>
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Stratified Split

Langkah 1 - Load Data

```
[29]
import pandas as pd

df2 = pd.read_csv('Titanic-Dataset-selected.csv')
df2.head()
```

✓ 0.0s

Hasil load data

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	115
1	1	1	0.638789	0	81
2	1	3	-0.284663	0	115
3	1	1	0.407926	0	55
4	0	3	0.407926	1	115

Langkah 2 - Split Data

```
# Split Data
from sklearn.model_selection import train_test_split

# Split data training dan lainnya
# Data lainnya, akan kita split lagi menjadi validasi dan testing
# Rasio yang akan kita gunakan adalah 8:1:1
df2_train, df2_unseen = train_test_split(df2, test_size=0.2, random_state=0, stratify=df['Survived'])

# Split lagi antara validasi dan testing
df2_val, df2_test = train_test_split(df2_unseen, test_size=0.5, random_state=0, stratify=df_unseen['Survived'])

# Cek masing-masing ukuran data

print(f'Jumlah label data asli:\n{df2.Survived.value_counts()}')
print(f'Jumlah label data train:\n{df2_train.Survived.value_counts()}')
print(f'Jumlah label data val:\n{df2_val.Survived.value_counts()}')
print(f'Jumlah label data test:\n{df2_test.Survived.value_counts()}')
```

[31] ✓ 0.0s

Hasilnya adalah sebagai berikut. Perhatikan rasio-rasio yang dihasilkan.

```
Jumlah label data asli:
Survived
0    549
1    342
Name: count, dtype: int64
Jumlah label data train:
Survived
0    439
1    273
Name: count, dtype: int64
Jumlah label data val:
Survived
0    51
1    38
Name: count, dtype: int64
Jumlah label data test:
Survived
0    59
1    31
Name: count, dtype: int64
```

Cross Validation 1

Pada praktikum ini, kita akan membuat data latih dan data uji saja dengan menggunakan metode cross validation.

Langkah 1 - Load Data

```
import pandas as pd

df3 = pd.read_csv('Titanic-Dataset-selected.csv')
df3.head()
```

[32] ✓ 0.0s

Hasil load data

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	115
1	1	1	0.638789	0	81
2	1	3	-0.284663	0	115
3	1	1	0.407926	0	55
4	0	3	0.407926	1	115

Langkah 2 - Split Data

```
# Implementasi k-fold cross validation (random) dengan training dan testing saja
from sklearn.model_selection import KFold

# inisiasi objek kfold
kf = KFold(n_splits=4)
print(f'Jumlah fold: {kf.get_n_splits()}')
print(f'Objek KFold: {kf}')

# Lakukan splitting dengan KFold
kf_split = kf.split(df3)
print(f'Jumlah data df: {df.shape[0]}')

# Cek indec data tiap fold
for train_index, test_index in kf_split:
    print(f'Index trainL {train_index} | Index Test: {test_index}')
```

[35] ✓ 0.0s

Perhatikan nomor index untuk setiap fold. Jika berhasil, maka seharusnya setiap fold memiliki nomor index yang berbeda untuk setiap data latih dan data uji.

Berikut hasilnya untuk 2 fold pertama.

```
Jumlah fold: 4
Objek KFold: KFold(n_splits=4, random_state=None, shuffle=False)
Jumlah data df: 891
Index trainL [223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240
 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258
 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276
 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294
 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312
 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330
 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348
 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366
 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384
 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402
 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420
 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438
 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456
 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474
 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492
 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510
 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528
 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546
 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564
 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582
 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600
 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618
...
 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848
 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866
 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884
 885 886 887 888 889 890]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Cross Validation 2

Pada pratikum ini, kita sedikit melakukan pendekatan yang berbeda dibandingkan dengan praktikum sebelumnya. Pertama data akan kita split menjadi data latih dan data uji. Kemudian, data latih kita split menjadi beberapa bagian dengan cross validation untuk kebutuhan data latih dan data validasi.

Langkah 1 - Load Data

```
import pandas as pd

df3 = pd.read_csv('Titanic-Dataset-selected.csv')
df3.head()
```

[32] ✓ 0.0s

Hasil load data

	Survived	Pclass	Age	Sex	Cabin
0	0	3	-0.592481	1	115
1	1	1	0.638789	0	81
2	1	3	-0.284663	0	115
3	1	1	0.407926	0	55
4	0	3	0.407926	1	115

Langkah 2 - Split Data

```
# Implementasi k-fold cross validation (random) dengan trining, validation, dan testing
from sklearn.model_selection import train_test_split, KFold

# Split dulu antara data training dan testing dengan train_test_split
# Rasio 8:2 untuk training dan testing
df4_train, df4_test = train_test_split(df4, test_size=0.2, random_state=0)

# inisiasi objek kfold
kf2 = KFold(n_splits=4)
print(f'Jumlah Fold: {kf2.get_n_splits()}')
print(f'Objek KFold: {kf2}')

# Lakukan splitting dengan KFold untuk data df_training
# Dengan cara ini, kita masih memiliki data testing untuk keperluan pengujian model
# namun tetap dapat melakukan evaluasi dengan menggunakan data validasi
kf2_split = kf2.split(df_train)
print(f'Jumlah data df_traing: {df4_train.shape[0]}')

# cek index data tiap fold
for train_index, test_index in kf2_split:
    print(f'Index train: {train_index} | Index test: {test_index}')
```

[40] ✓ 0.0s

Hasilnya, pada jumlah data latih seharusnya berkurang karena telah kita split pada tahap awal.

Berikut merupakan hasil untuk 2 fold pertama.

```
Jumlah Fold: 4
Objek KFold: KFold(n_splits=4, random_state=None, shuffle=False)
Jumlah data df_traing: 712
Index train: [178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195
 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213
 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231
 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249
 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267
 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285
 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303
 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321
 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339
 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357
 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375
 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393
 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411
 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429
 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447
 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465
 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483
 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501
 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519
 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537
 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555
 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573
...
 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659
 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677
 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695
 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Praktikum 3d
Ekstraksi Fitur Data Tidak Terstruktur

Pada pratikum ini, kita akan mencoba untuk mengekstraksi fitur dari sebuah citra. Proses yang akan kita lakukan hanya proses sederhana. Pembahasan lengkap terkait dengan pra pengolahan data dan ekstraksi fitur pada data citra akan dibahas pada perkuliahan pengolahan citra digital. Data yang akan kita gunakan adalah citra berikut.

Langkah 0 - Instal Library

Pada praktikum ini kita akan menggunakan library pillow. Tambahan pillow terlebih dahulu apabila belum terinstal didalam perangkat yang Anda gunakan.

Jalan perintah berikut pada notebook

```
!pip install Pillow
```

```
[41] ✓ 10.2s
```

```
Requirement already satisfied: Pillow in D:\Aplikasi\anaconda3\Lib\site-packages (12.3.0)
```

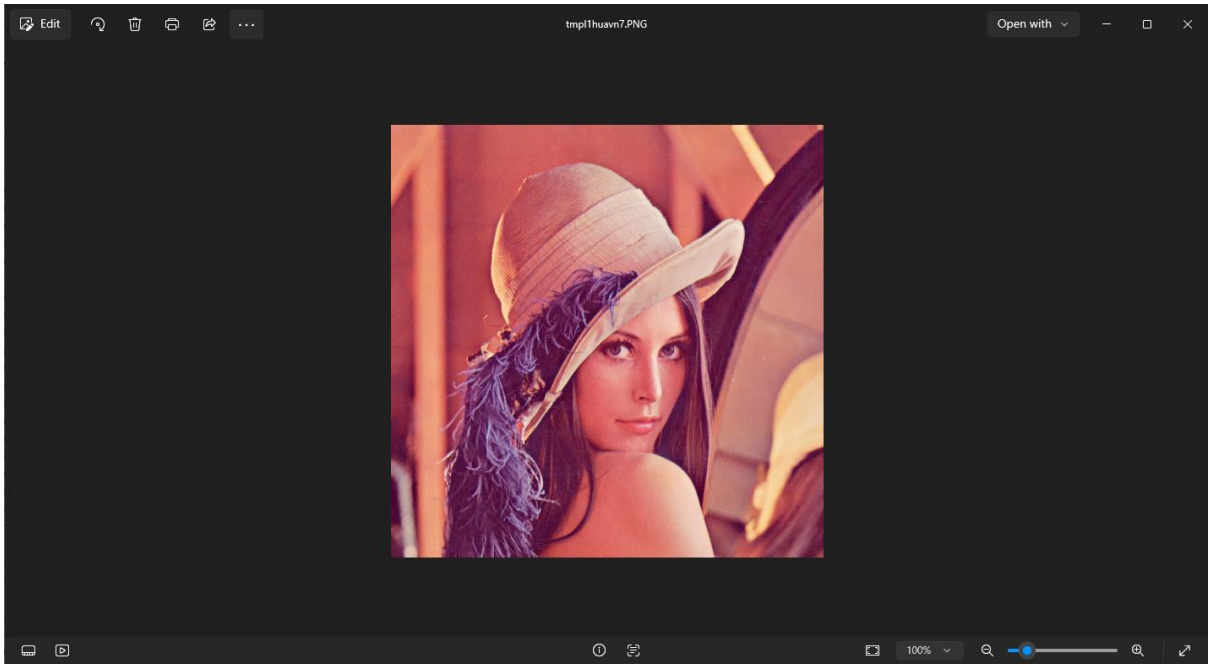
Langkah 1 - Load Image

```
from PIL import Image

img = Image.open('Lenna.png')
img.show() #tampilkan gambar
display(img) # metode alternatif tampilkan gambar
```

```
[42] ✓ 1.5s
```





Langkah 2 - Ekstrak Fitur

```
# Ekstrak setiap channel red, green, blue
r, g, b = img.split()

# Cek panjang ukuran channel red
print(len(r.histogram()))

# Cetak fitur histogram pada channel red
print(r.histogram())
```

[43] ✓ 0.0s

Anda akan mendapatkan nilai histogram dalam bentuk array

```
256
[0, 0, 0, 
```

TUGAS PRAKTIKUM

Pada tugas pratikum ini Anda akan menggunakan data "Wisconsin Breast Cancer". Data tersebut terdiri dari 569 data yang digunakan untuk mendiagnonis jenis kanker **Malignant (M) dan Benign (B)**. Tugas Anda adalah,

1. **Pisahkan antara variabel yang dapat digunakan dan variabel yang tidak dapat digunakan.**
   Jawab :
   Load data:

```
import pandas as pd

dpath = '../Data Praktikum/wbc.csv'
df = pd.read_csv(dpath)
df.head()
```

Hasil:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67

5 rows × 33 columns

perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst	Unnamed: 32
184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890	NaN
158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902	NaN
152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758	NaN
98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300	NaN
152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678	NaN

Pisah variabel yang diguanakn dan tidak dapat digunakan

```
df_simpan = df['diagnosis']

df_drop = df.drop(columns=['id', 'diagnosis', 'Unnamed: 32'])
df_drop.head()
```

Disini saya menyimpan variabel diagnonis untuk proses encoding pada kolom diagnosis dan menyimpannya pada variabel ***df_simpan.***
Lalu dibawahnya saya menghapus 3 variabel, yakni id, diagnosis, dan Unnamed: 32 karena variabel tersebut tidak digunakan dan menyimpannya pada variabel ***df_drop***.
Setelahnya saya menampilkan data dengan 3 variabel yang sudah di hapus tersebut menggunakan ***df_drop.head()***

Hasil:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	fractal_dimension_mean
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883

5 rows × 30 columns

radius_worst	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst
25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300
22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678

2. **Lakukan proses encoding pada kolom "diagnosis".**

Jawab:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df_simpan_encoded = le.fit_transform(df['diagnosis'])
```

✓ 0.0s

Import library LabelEncoder dari sklearn.preprocessing untuk proses Encoding pada kolom ***"diagnosis".***

Selanjutnya saya membuat objek dari LabelEncoder dengan nama ***le***.

Selanjutnya saya membuat variabel ***df_simpan_encoded*** untuk menyimpan hasil dari proses encoding pada kolom **"*diagnosis*".**

Hasil Encoding:

```
df_simpan_encoded[:5]
```

✓ 0.0s

```
array([1, 1, 1, 1, 1])
```

Disini saya tidak menggunakan ***df_simpan_encoded.head()*** karena numpy array menggunakan slicing bukan method ***.head().***

3. **Lakukan proses standardisasi pada semua kolom yang memiliki nilai numerik.**

Jawab:

```
from sklearn.preprocessing import StandardScaler

std = StandardScaler()
df_drop_std = std.fit_transform(df_drop)
```

✓ 0.0s

Import library StandardScaler dari sklearn.preprocessing untuk proses standarisasi pada kolom ***"numerik"***

Selanjutnya saya membuat objek dari StandardScaler dengan nama ***std.***

Selanjutnya saya membuat variabel ***df_drop_std*** untuk menyimpan hasil dari proses standarisasi pada kolom numerik. *(Karena pada variabel df_drop semua kolom berisi numerik maka akan langung dilakukan proses standarisasi)

Hasil Standarisasi:

```
df_drop_std[:3, :30]
```
✓ 0.0s

```
array([[ 1.09706398e+00, -2.07333501e+00,  1.26993369e+00,
         9.84374905e-01,  1.56846633e+00,  3.28351467e+00,
         2.65287398e+00,  2.53247522e+00,  2.21751501e+00,
         2.25574689e+00,  2.48973393e+00, -5.65265059e-01,
         2.83303087e+00,  2.48757756e+00, -2.14001647e-01,
         1.31686157e+00,  7.24026158e-01,  6.60819941e-01,
         1.14875667e+00,  9.07083081e-01,  1.88668963e+00,
        -1.35929347e+00,  2.30360062e+00,  2.00123749e+00,
         1.30768627e+00,  2.61666502e+00,  2.10952635e+00,
         2.29607613e+00,  2.75062224e+00,  1.93701461e+00],
       [ 1.82982061e+00, -3.53632408e-01,  1.68595471e+00,
         1.90870825e+00, -8.26962447e-01, -4.87071673e-01,
        -2.38458552e-02,  5.48144156e-01,  1.39236330e-03,
        -8.68652457e-01,  4.99254601e-01, -8.76243603e-01,
         2.63326966e-01,  7.42401948e-01, -6.05350847e-01,
        -6.92926270e-01, -4.40780058e-01,  2.60162067e-01,
        -8.05450380e-01, -9.94437403e-02,  1.80592744e+00,
        -3.69203222e-01,  1.53512599e+00,  1.89048899e+00,
        -3.75611957e-01, -4.30444219e-01, -1.46748968e-01,
         1.08708430e+00, -2.43889668e-01,  2.81189987e-01],
       [ 1.57988811e+00,  4.56186952e-01,  1.56650313e+00,
         1.55888363e+00,  9.42210440e-01,  1.05292554e+00,
         1.36347845e+00,  2.03723076e+00,  9.39684817e-01,
        -3.98007910e-01,  1.22867595e+00, -7.80083377e-01,
         8.50928301e-01,  1.18133606e+00, -2.97005012e-01,
         8.14973504e-01,  2.13076435e-01,  1.42482747e+00,
         2.37035535e-01,  2.93559404e-01,  1.51187025e+00,
        -2.39743838e-02,  1.34747521e+00,  1.45628455e+00,
         5.27407405e-01,  1.08293217e+00,  8.54973944e-01,
         1.95500035e+00,  1.15225500e+00,  2.01391209e-01]])
```

Disini saya tidak menggunakan ***df_drop_std.head()*** karena numpy array menggunakan slicing bukan method ***.head().*** Saya juga menampilkan data yang sudah di Standarisasi pada 3 baris 30 kolom.

4. **Lakukan proses stratified split data untuk membuat data latih dan data uji dengan rasio 80:20.**

   Jawab:

   Import library ***train_test_split*** dari ***sklearn.model_selection*** untuk membagi data menjadi train dan test. Import library ***pandas*** untuk mengubah numpy array menjadi Series.

   Selanjutnya saya membagi data menjadi 80% training dan 20% unseen menggunakan ***train_test_split*** dengan parameter ***test_size=0.2***. Parameter ***stratify=df_simpan_encoded*** digunakan untuk menjaga proporsi label tetap sama.

   Selanjutnya saya membagi data unseen (20%) menjadi validation dan test dengan rasio 50:50 menggunakan parameter ***test_size=0.5***. Hasilnya adalah 10% validation dan 10% test dari total data.

   Selanjutnya saya mengubah numpy array menjadi pandas Series agar bisa menggunakan method ***.value_counts()*** untuk menghitung distribusi label.

   Selanjutnya saya menampilkan jumlah data dan distribusi label ***(0=Benign, 1=Malignant)*** pada setiap subset untuk memverifikasi bahwa pembagian data berhasil dengan proporsi yang seimbang.

```python
from sklearn.model_selection import train_test_split
import pandas as pd

# Split pertama: 80% train, 20% unseen
df_drop_train, df_drop_unseen, df_simpan_train, df_simpan_unseen = train_test_split(
    df_drop_std,
    df_simpan_encoded,
    test_size=0.2,
    random_state=0,
    stratify=df_simpan_encoded
)

# Split kedua: bagi unseen jadi val dan test
df_drop_val, df_drop_test, df_simpan_val, df_simpan_test = train_test_split(
    df_drop_unseen,
    df_simpan_unseen,
    test_size=0.5,
    random_state=0,
    stratify=df_simpan_unseen
)

# Ubah numpy array jadi pandas Series
df_simpan_encoded_series = pd.Series(df_simpan_encoded, name='diagnosis')
df_simpan_train_series = pd.Series(df_simpan_train, name='diagnosis')
df_simpan_val_series = pd.Series(df_simpan_val, name='diagnosis')
df_simpan_test_series = pd.Series(df_simpan_test, name='diagnosis')

print('UKURAN DATA')
print(f'Jumlah data asli: {len(df_simpan_encoded)}')
print(f'Jumlah data train: {len(df_simpan_train)}')
print(f'Jumlah data val: {len(df_simpan_val)}')
print(f'Jumlah data test: {len(df_simpan_test)}')

print('DISTRIBUSI LABEL')
print(f'\nJumlah label data asli:\n{df_simpan_encoded_series.value_counts()}')
print(f'\nJumlah label data train:\n{df_simpan_train_series.value_counts()}')
print(f'\nJumlah label data val:\n{df_simpan_val_series.value_counts()}')
print(f'\nJumlah label data test:\n{df_simpan_test_series.value_counts()}')
```

Hasil Stratified Split:

```
UKURAN DATA
Jumlah data asli: 569
Jumlah data train: 455
Jumlah data val: 57
Jumlah data test: 57
DISTRIBUSI LABEL

Jumlah label data asli:
diagnosis
0    357
1    212
Name: count, dtype: int64

Jumlah label data train:
diagnosis
0    285
1    170
Name: count, dtype: int64

Jumlah label data val:
diagnosis
0    36
1    21
Name: count, dtype: int64
...
diagnosis
0    36
1    21
Name: count, dtype: int64
```